

BaseSim : Plateforme de simulation de transmission numérique en bande de base

BaseSim est une application Python destinée aux ingénieurs et étudiants en télécommunications pour simuler des chaînes de transmission numérique en bande de base. Elle permet de convertir une séquence binaire en signal modulé (par exemple BPSK, QPSK, QAM), d'y ajouter du bruit (AWGN) ou d'autres perturbations, puis de démoduler et de décoder le signal pour comparer les bits reçus aux bits envoyés. Cette plateforme pédagogique illustre comment la modulation, les caractéristiques du canal et les codes correcteurs influencent la qualité de la transmission. Les concepts sont basés sur la théorie des communications numériques : par exemple, « la modulation en bande de base module des symboles de transmission numérique en ondes sinusoïdales », et le canal AWGN (Additive White Gaussian Noise) est un modèle de bruit fondamental en théorie de l'information. BaseSim fournit ainsi un environnement simple pour expérimenter ces principes et observer graphiquement les résultats.

Fonctionnalités principales

- **Modulations numériques** : BaseSim inclut plusieurs schémas de modulation en bande de base couramment étudiés, tels que la modulation d'amplitude en impulsion (PAM), PSK (BPSK, QPSK, 8-PSK, etc.), QAM (QAM16, QAM64, ...), etc. La modulation transforme les bits en symboles numériques selon la méthode choisie.
- **Canaux de transmission** : L'application modélise principalement un canal à bruit blanc gaussien additif (AWGN), où le bruit est ajouté au signal modulé. On peut régler le rapport signal-sur-bruit (SNR) pour simuler des conditions réelles. D'autres canaux simples (par exemple BSC binaire) peuvent être prévus.
- **Codage de canal** : BaseSim propose des options de codage et décodage canal pour étudier la correction d'erreurs. Par exemple, on peut activer un code de type Hamming, Reed-Solomon, ou code convolutionnel avec décodage Viterbi. Ces codes ajoutent de la redondance aux bits émis afin de « contrôler les erreurs dans la transmission » sur un canal bruyant.
- **Analyse des performances** : Après simulation, BaseSim calcule le taux d'erreur binaire (BER) en comparant bits envoyés et bits reçus. On peut ainsi observer comment le BER évolue en fonction du SNR et des paramètres choisis.
- **Affichage graphique** : Les signaux (onde modulée, constellation, diagramme de l'œil, etc.) sont tracés avec Matplotlib pour visualiser la transmission. Par exemple, on peut générer un diagramme de l'œil pour observer l'ouverture du bit (voir figure ci-dessous) ou tracer l'évolution temporelle du signal modulé.

Technologies utilisées

- **Python 3.8+** : BaseSim est développé en Python, version 3.8 ou ultérieure. Python est un langage très utilisé en ingénierie et scientifique. Ses bibliothèques standard et tierces facilitent le traitement numérique.
- **Tkinter** : l'interface graphique est construite avec Tkinter, le **gui standard de Python**, simple et multiplateforme. Tkinter fournit boutons, champs de saisie et menus pour interagir avec l'application.

- **NumPy** : pour les calculs numériques sur tableaux (génération de données, opérations mathématiques). NumPy offre des objets de type `ndarray` et de nombreuses fonctions mathématiques optimisées, essentiels pour simuler des signaux numériques.
- **SciPy** : pour des algorithmes scientifiques avancés (filtres, transformées, calculs statistiques, optimisations...). Par exemple, des fonctions de filtrage peuvent servir à façonner les impulsions ou à démoduler le signal.
- **Matplotlib** : pour tracer les courbes de signal, constellation ou diagramme de l'œil. C'est une bibliothèque de visualisation complète qui intègre animations et sauvegarde d'images.
- **Autres** : la bibliothèque standard (ex. `csv`, `math`) et éventuellement `pandas` pour exporter les résultats. Tkinter est inclus de base avec Python, mais sur certains systèmes Linux il peut être nécessaire d'installer le paquet `python3-tk`.

Installation

Pour installer BaseSim et ses dépendances, procédez comme suit :

1. **Installer Python 3.8 ou supérieur.** Téléchargez et installez Python depuis python.org ou utilisez votre gestionnaire de paquets. Vérifiez avec `python --version` ou `python3 --version`.
2. **Installer les bibliothèques requises.** Ouvrez une console et lancez :
3. `pip install numpy scipy matplotlib`

Si vous utilisez un environnement virtuel (recommandé), activez-le avant. Tkinter est fourni avec Python ; sur certains systèmes, installez `python3-tk` (Debian/Ubuntu) ou équivalent.

4. **Télécharger BaseSim.** Clonez le dépôt Git (ou téléchargez l'archive) du projet BaseSim.
5. **Configurer l'environnement.** Assurez-vous d'avoir une connexion Internet pour télécharger les paquets (ou utilisez `conda` si vous préférez). Après installation, vérifiez que l'exemple de Tkinter s'ouvre :
6. `python -m tkinter`

Si une fenêtre de démo s'affiche, Tkinter est correctement installé.

Exécution

1. **Lancer l'application** : dans un terminal, positionnez-vous dans le dossier BaseSim et exécutez la commande principale, par exemple `python basesim.py` ou `python main.py` (selon le nom du script principal). Une interface graphique Tkinter apparaît alors.
2. **Interface principale** : vous disposez d'une zone de saisie pour les bits, de menus déroulants pour la modulation, le canal, le codage et d'un champ numérique pour le SNR, ainsi qu'un bouton « Démarrer ».
3. **Lancer une simulation** : après avoir saisi les paramètres (voir **Guide utilisateur** ci-dessous), cliquez sur **Démarrer**. L'application génère la séquence modulée, y ajoute du bruit selon le SNR spécifié, puis effectue la démodulation et le décodage. Les résultats (bits reçus, BER, signaux tracés) s'affichent ensuite.

Exemple de simulation

Imaginons que nous voulons tester une transmission BPSK avec un canal AWGN. Voici un exemple d'utilisation :

1. Dans la zone « **Saisie des bits** », entrez la trame de bits, par exemple 1 0 1 1 0 0 1.
2. Choisissez **Modulation : BPSK**, **Canal : AWGN**, **Codage : Aucun** (ou un code Hamming si disponible), et **SNR** par exemple 10 dB.
3. Cliquez sur **Démarrer**. BaseSim va moduler la trame, ajouter du bruit et démoduler.
4. La figure du signal modulé s'affiche ainsi que le diagramme de l'œil. Le programme affiche le nombre de bits erronés et le BER. Par exemple, pour un SNR élevé on obtient peu d'erreurs, tandis que pour un SNR faible le BER augmente.
5. Vous pouvez répéter avec d'autres paramètres (par exemple QAM16, code convolutionnel avec Viterbi, SNR réduit) pour observer l'impact sur la courbe BER.

Cet exemple illustre comment les paramètres de la chaîne numérique affectent la performance en termes d'erreur et de qualité du signal.

Liens utiles

- **Documentation Python** : guide officiel Python (classes, syntaxe, bibliothèques) – voir notamment [doc Tkinter en français](#).
- **NumPy** : site officiel [numpy.org](#) – « NumPy est le package fondamental pour le calcul scientifique en Python ».
- **SciPy** : site [scipy.org](#) – SciPy fournit des algorithmes pour l'optimisation, l'intégration, l'interpolation....
- **Matplotlib** : site [matplotlib.org](#) – Matplotlib est une bibliothèque complète pour créer des visualisations en Python.
- **Communications Numériques** : tutoriels ou références en ligne sur la modulation (par exemple [Digital Baseband Modulation – MathWorks](#)) ou sur les codes correcteurs (voir [Error correction code – Wikipédia](#)).
- **Pandoc** : outil de conversion de documents (Markdown vers PDF/Word/HTML). On peut générer un PDF à partir du README.md avec Pandoc (par exemple `pandoc -s -o BaseSim.pdf README.md`) car « Pandoc... peut convertir des documents Markdown en PDF, HTML, DOCX, etc. ».

Guide utilisateur (interface graphique)

La section suivante décrit l'utilisation de BaseSim via l'interface Tkinter, étape par étape. Les captures d'écran illustrent les principaux éléments (pour cette documentation, les images proviennent d'exemples types et non d'une interface réelle, mais elles représentent l'aspect général attendu).

Saisie des bits

L'utilisateur saisit la **séquence binaire** à transmettre dans le champ dédié. Par exemple, on peut entrer une chaîne comme 10110011. Les bits sont généralement séparés par des espaces

ou concaténés selon le format prévu. Ce champ permet de tester différentes trames d'information.

Choix des paramètres

- **Modulation** : choisissez le schéma de modulation numérique (BPSK, QPSK, 8-PSK, QAM16, QAM64, PAM4, etc.) via un menu déroulant. Ce paramètre détermine comment les bits seront convertis en symbole par le modulateur.
- **Canal de transmission** : sélection du modèle de canal. Par exemple, **AWGN** pour un bruit blanc gaussien additif (avec une option de réglage du **SNR** en dB). Le champ SNR à saisir (ex. 5, 10 ou plus) définit la puissance du signal relative au bruit.
- **Codage de canal** : choix d'un code correcteur d'erreurs (aucun, code de Hamming (7,4), code convolutionnel, etc.). L'activation du codage ajoute des bits de parité lors de l'émission et utilise un décodeur adapté au récepteur.
- **Paramètres divers** : d'autres réglages peuvent exister (nombre de bits, filtrage, fréquence d'échantillonnage, etc.), selon les fonctionnalités de BaseSim. L'interface doit regrouper ces options clairement.

Après avoir configuré ces paramètres, l'utilisateur clique sur le bouton **Démarrer** (ou « Lancer la simulation ») pour exécuter la simulation avec les paramètres sélectionnés.

Lancement de la simulation

Lorsqu'on clique sur **Démarrer**, BaseSim réalise les opérations suivantes en arrière-plan :

- Génération ou importation des **bits** entrés.
- **Codage** de ces bits si activé (par ex. Hamming, convolutionnel).
- **Modulation** du signal : les bits sont transformés en vecteurs complexes selon la modulation choisie (par exemple, BPSK mappe $0 \rightarrow +1$, $1 \rightarrow -1$).
- **Filtrage** éventuel : application d'un filtre de mise en forme (par ex. filtre en racine sinusoïdale surélevée).
- **Ajout de bruit** : le signal modulé est passé dans le canal AWGN, en fonction du SNR spécifié.
- **Démodulation** : réception du signal bruité, puis démodulation selon la même modulation (et décodage de canal si applicable).
- Calcul du **taux d'erreurs (BER)** en comparant bits émis et bits reçus après décodage.

Une fois le calcul terminé, l'interface affiche les résultats (voir ci-dessous).

Affichage des signaux et bits

Après simulation, BaseSim affiche plusieurs graphiques et informations dans l'interface, par exemple :

- **Signal modulé en sortie de modulateur** (tracé temporel).
- **Constellation** (nuage de points I/Q après modulation, si applicable).
- **Diagramme de l'œil** du signal reçu (pour PSK/PAM), permettant d'observer l'ouverture de l'œil et la qualité temporelle du signal (voir figure).

Figure : Exemple de signal modulé (ici sinusoïdal par commodité) affiché par Matplotlib après simulation. Chaque cycle de sinusoïde correspond à un bit transmis. Les transitions abruptes entre niveaux (0→1) représentent l'enchaînement des bits.

Figure : Exemple de diagramme de l'œil généré après démodulation (pour une transmission BPSK ou QPSK). Un « œil » bien ouvert (espaces blancs visibles) indique peu de distorsion et un faible BER, tandis qu'un œil fermé signifie un signal bruité.

En plus des graphiques, les **bits reçus** après démodulation sont affichés (par exemple dans une zone texte ou un fichier de log), ainsi que le BER calculé. L'utilisateur peut ainsi comparer visuellement et numériquement l'effet des paramètres choisis.

Exportation des résultats

BaseSim permet d'**exporter** les résultats de la simulation pour analyse ultérieure :

- **Sauvegarde des données** : clic sur un bouton « Exporter » pour enregistrer les bits transmis et reçus, le BER, etc., dans un fichier texte ou CSV. Par exemple, on peut obtenir un fichier `resultats.csv` contenant la trame émise, la trame reçue et l'erreur binaire.
- **Sauvegarde des graphiques** : les figures tracées par Matplotlib peuvent être exportées (par exemple via `plt.savefig`) en formats image (PNG, PDF) depuis l'interface.
- **Rapport PDF** : certains outils Markdown comme Pandoc permettent de générer un rapport PDF complet reprenant les graphes et les données. On peut ainsi documenter chaque simulation.

L'exportation des résultats aide à générer des rapports ou à poursuivre l'analyse dans d'autres logiciels (ex. Excel, MATLAB).

Génération de la documentation au format PDF

La documentation fournie ci-dessus est également disponible au format PDF pour une consultation hors-ligne. On peut obtenir le PDF en convertissant ce même contenu Markdown via Pandoc (outil de conversion universel). Par exemple, la commande suivante génère un PDF à partir du fichier `README.md` :

```
pandoc -s -o BaseSim.pdf README.md
```

Pandoc permet en effet de convertir facilement des documents Markdown en formats PDF, HTML, DOCX, etc.. Il requiert un processeur LaTeX (ex. TeX Live) pour produire le PDF final. Cela garantit une mise en page propre reprenant la structure (titres, listes, figures) du README.

Remarque : Les exemples d'images ci-dessus (signal sinusoïdal, diagramme de l'œil) illustrent les types de graphiques que BaseSim peut produire. Dans l'interface réelle, les graphes seraient générés dynamiquement selon les données simulées. Les captures ci-dessus

proviennent de ressources de démonstration montrant ce que Matplotlib peut tracer en contexte de communications numériques.